# Sistemas Operacionais - Relatório da Atividade 3

**Virtualização, Contêineres, Namespaces e Cgroups**

Período: 2026.2

## 1. Contexto Teórico e Pergunta Norteadora

O presente relatório documenta a execução de um laboratório reprodutível cujo objetivo é observar na prática os mecanismos de isolamento de processos e a limitação de recursos em contêineres Linux. Diferente do desenvolvimento de uma aplicação de ponta a ponta, o foco deste experimento está na infraestrutura do Sistema Operacional (SO) subjacente, analisando como o kernel expõe e regula os recursos computacionais sem a necessidade de emulação de hardware subjacente.

Na virtualização tradicional, um hipervisor (seja do Tipo 1, *bare-metal*, ou do Tipo 2, hospedado) é responsável por emular os dispositivos de hardware para que múltiplos sistemas operacionais hóspedes possam coexistir. Essa abordagem, embora segura, introduz uma sobrecarga computacional considerável. A conteinerização, por outro lado, emprega uma abordagem de virtualização em nível de sistema operacional. Todos os contêineres executados em um host compartilham exatamente o mesmo kernel, utilizando estruturas de dados lógicas para criar a ilusão de isolamento. É fundamental compreender que um contêiner não é uma entidade de primeira classe no kernel Linux; ele é uma abstração construída em espaço de usuário (*user space*) por meio da combinação de funcionalidades nativas do kernel, primordialmente *namespaces* e *cgroups*.

> **Pergunta Norteadora Central:** Como namespaces alteram a visão de processos, mounts e rede, e como cgroups limitam recursos sem transformar um contêiner em uma máquina virtual independente?

Para responder a essa questão de forma empírica, a metodologia empregada exigiu a coleta de evidências reais e brutas através do motor Docker, analisando a distinção dos *namespaces* e submetendo uma carga de trabalho sintética e estritamente controlada para validar e quantificar o comportamento dos *Control Groups* (cgroups).

## 2. Hipóteses de Trabalho e Comportamento Esperado

Com base na fundamentação teórica sobre virtualização baseada em contêineres e documentação técnica do kernel Linux, foram formuladas três hipóteses principais a serem verificadas pelas medições do laboratório:

- **H1 (Isolamento de Visão via Namespaces):** Dois contêineres iniciados a partir da exata mesma imagem de disco (rootfs) possuirão *handles* de namespace (PID, mount e net) diferentes entre si. Como consequência desse particionamento lógico, ambos apresentarão um processo que acredita possuir o PID 1 interno. No entanto, do ponto de vista do host (o verdadeiro árbitro dos recursos), esses processos mapearão para PIDs completamente distintos e sequenciais na tabela global do kernel.
- **H2 (Gestão de Recursos via Cgroups):** Ao aplicar um trabalho de CPU determinístico, fixo e isolado em uma única thread (sem paralelismo), a imposição de uma quota de 0,5 CPU (Condição C1) fará com que o tempo de parede (*wall time*) seja aproximadamente o dobro daquele observado com uma quota de 1,0 CPU (Condição C2). Em C1, o fenômeno de *throttling* (estrangulamento térmico/orçamentário imposto pelo escalonador do kernel) será amplamente visível nos contadores do `cpu.stat`, evidenciando pausas forçadas.
- **H3 (Princípio do Menor Privilégio e Proteção de Mount):** A montagem de um diretório sintético do host para o contêiner com a flag de "somente leitura" (*read-only*) garantirá o acesso de leitura, mas interceptará e negará tentativas de escrita no nível do barramento de montagem. Essa negação ocorrerá de forma independente das permissões clássicas de arquivo (POSIX) do hospedeiro, configurando uma camada adicional de defesa profunda (*defense-in-depth*).

## 3. Arquitetura Físico-Lógica e Ambiente de Execução

O experimento utilizou um ambiente Linux nativo rigorosamente inventariado. A utilização de um sistema bare-metal foi imperativa para garantir que as observações relativas ao kernel (como o escalonamento de CPU e a medição de tempo de parede) não sofressem interferência não documentada de uma camada de hipervisor intermediária ou de traduções de chamadas de sistema (como ocorre no WSL2 para Windows). A tabela a seguir descreve as especificações técnicas da infraestrutura utilizada para sustentar o laboratório, coletadas em 07/11/2026.

| Componente | Especificação Registrada |
|---|---|
| **Ambiente Linux real do daemon** | Linux em máquina física *desktop*, sem máquina virtual (VM) intermediária. Esta condição foi inferida pois o utilitário `lscpu` lista diretamente o modelo físico da CPU e não apresenta linhas referentes a hipervisores. O daemon local foi acessado via `sudo docker`. |
| **Distribuição e Kernel** | Ubuntu 24.04.2 LTS (Noble Numbat). Kernel Linux versão 7.0.0-38-generic #38~24.04.4-Ubuntu SMP PREEMPT_DYNAMIC (arquitetura x86_64). |

| Componente | Especificação Registrada |
| --- | --- |
| **Hardware (Processamento)** | Processador AMD Ryzen 5 5600GT. Contém 1 socket físico, 6 núcleos e 2 threads por núcleo, totalizando 12 CPUs lógicas disponíveis para o sistema (confirmado via comando `nproc = 12`). O boost de frequência estava ativado dinamicamente pelo kernel (governador de energia), com escala operando em 75% no momento do inventário (mínimo de 421,9 MHz e teto máximo de 4668,75 MHz). |
| **Memória e Swap** | A máquina hospedeira possui 15 GiB de Memória RAM totais (sendo que 5,1 GiB estavam em uso por outros processos do SO e interface gráfica no momento do inventário, com 10 GiB livres). A partição de *swap* ativa possuía 4,0 GiB de capacidade. |
| **Motor de Contêineres** | Docker versão 29.1.3 (build 29.1.3-0ubuntu3~24.04.2) integrado ao Compose versão 2.40.3+ds1-0ubuntu1~24.04.1. |
| **Mecanismo de Cgroups** | Hierarquia unificada (cgroup v2) gerenciada através do driver `systemd`. Controladores habilitados: `cpuset cpu io memory hugetlb pids rdma misc dmem`. |
| **Imagem e Payload Base** | A imagem utilizada para instanciar os contêineres foi fixada por segurança via digest criptográfico: `python@sha256:4840d514c583a4d3d8cad3940e68dee6f0f6f3635c12fcd0a3d74c21a5b93a54`. Internamente, esta imagem fornece o interpretador Python 3.12.16. O código de estresse sintético adotado (`scripts/carga_cpu.py`) está rastreado pelo sha256 `b1165487d16ee249d4838cc1dc9e103579446da16236760a1f6dfca8edaf1550`. A rodada oficial encontra-se no commit `6704f2eb7d1b76d378233f054261604e7224d88b`. |

Por tratar-se de um ambiente local onde o terminal que despacha os comandos compartilha o exato mesmo núcleo do daemon Docker em segundo plano, os PIDs exibidos pela ferramenta de inspeção do Docker (via `docker inspect`) correspondem intrinsecamente ao espaço de processos (tabela localizada em `/proc`) do hospedeiro ativo. Diferentemente do uso do Docker Desktop em sistemas Windows ou macOS (onde há uma VM Linux transparente emulando o kernel hospedeiro), neste laboratório a ponte entre o contêiner e o hardware é direta.

# 4. Resultados da Coleta e Evidências Metodológicas

## 4.1. Isolamento de Processos (Análise de Namespaces)

A primeira bateria de testes focou na análise estrutural de processos. Foram executados de modo concorrente dois contêineres independentes ( `av3-ns-a` e `av3-ns-b` ), gerados a partir da mesma imagem (digest fixo) e executando o mesmo comando ocioso ( `sleep 3600` ). Para eliminar ruídos de comunicação externa, ambos foram desprovidos de interface de rede padronizada utilizando a flag `--network none` . Durante o ciclo de vida ativo, comandos `docker inspect` e `docker exec` foram utilizados para perfilar a numeração de PIDs no host e investigar a estrutura de arquivos pseudo-virtuais expostos em `/proc/1/ns/` dentro do limite dos contêineres.

| **Recurso / Artefato Avaliado** | **Handle coletado em A (av3-ns-a)** | **Handle coletado em B (av3-ns-b)** | **Interpretação e Implicação no Kernel** |
|---|---|---|---|
| **PID mapeado no Host (inspect)** | 39466 | 39524 | Resultados **diferentes**. Para o kernel monolítico unificado, tratam-se simplesmente de dois processos distintos. Cada um possui um PID próprio e exclusivo acompanhando a numeração nativa do host. |
| **Nome do Processo PID 1 Interno** | `sleep` | `sleep` | Resultados **iguais**. Cada contêiner recebe seu próprio PID namespace da syscall `clone()` ou `unshare()` , permitindo que a primeira tarefa invocada assuma o cobiçado posto de PID 1 dentro daquele contexto restrito. A semelhança nominal deriva da execução do mesmo binário oriundo de imagens idênticas. Contudo, como provado acima (39466 vs 39524), *não se trata do mesmo processo em hardware*. |
| **Handle de Processos (/proc/1/ns/pid)** | pid: [4026532892] | pid: [4026532964] | Resultados **diferentes**. Confirma-se o isolamento estrutural: cada contêiner aponta para uma instância distinta de *PID namespace*, impossibilitando que os processos de "A" interajam com os sinais POSIX (ex: SIGTERM, SIGKILL) dos processos em "B". |
| **Handle de Montagem (/proc/1/ns/mnt)** | mnt: [4026532880] | mnt: [4026532961] | Resultados **diferentes**. Indicativo de tabelas de montagem estritamente independentes. O rootfs de "A" pode sofrer mutações sem que a hierarquia de diretórios de "B" sofra qualquer interferência, sendo este o pilar principal que simula um disco rígido isolado. |

| Recurso / Artefato Avaliado | Handle coletado em A (av3-ns-a) | Handle coletado em B (av3-ns-b) | Interpretação e Implicação no Kernel |
|---|---|---|---|
| **Handle de Rede (/proc/1/ns/net)** | net: [4026532894] | net: [4026532966] | Resultados **diferentes**. As pilhas lógicas de rede (interfaces, tabelas de roteamento e regras de iptables) estão apartadas. Vale destacar uma importante limitação analítica: como ambos contêineres usam a flag `--network none`, o fato de os handles serem distintos indica separação da estrutura no kernel, mas não implica conectividade de rede efetiva. |

É salutar ressaltar o que estas evidências precisas **não demonstram**. Os *handles* expostos apenas atestam que os processos foram acomodados em instâncias segregadas de namespace. Eles não constituem prova cabal de que o isolamento perimetral seja intransponível ou completamente livre de vulnerabilidades (*container breakout*). Eles não abordam namespaces periféricos fundamentais (como *user*, *uts* e *ipc*) que não foram medidos neste roteiro, e sobretudo, **não impõem limites rigorosos sobre o consumo excessivo de recursos de hardware**. Uma contenção baseada apenas em namespaces falharia de imediato caso o processo consumisse toda a CPU disponível no host.

## 4.2. Avaliação de Desempenho e Metodologia de Estresse Computacional

A imposição de limites precisos sobre recursos é orquestrada por outra ferramenta do kernel: os *Control Groups* (Cgroups). Para avaliar esse mecanismo, desenvolveu-se uma carga de trabalho analítica e estritamente determinística (script `carga_cpu.py`). A operação matemática escolhida não foi uma inferência de LLM complexa e impura, mas sim a execução repetitiva do algoritmo de derivação de chave criptográfica PBKDF2-HMAC-SHA256.

A carga foi calibrada para processar exatas 20.000.000 de iterações, fracionadas em blocos de verificação a cada 100.000 loops. O uso de uma senha fixa e de um "sal" (salt) derivado de índice gerou resumos (digests) progressivos encadeados em um hash SHA-256 definitivo que funciona como *checksum* de corretude. O trabalho é efetuado integralmente dentro de uma única thread (núcleo), o que permite correlacionar puramente a fração de tempo de CPU recebida com o tempo real de relógio (*wall time*), isentando a experiência de condições de corrida (*race conditions*) ou paralelizacão complexa. A carga é autolimitada internamente por um temporizador (sinal após 60 s) e envolvida num *GNU timeout* (90 s), garantindo que o hospedeiro estaria protegido em caso de bloqueio irreversível.

Foi adotada a alternância sistemática das duas condições principais do experimento com 3 repetições (C1-1, C2-1, C1-2, C2-2, C1-3, C2-3), incluindo uma pausa estabilizadora de 1 segundo entre as requisições. As condições testadas foram as seguintes:

- **Condição C1 (Meia CPU):** Imposição da quota de 0,5 CPU por meio de parâmetros de controle em cgroups.
- **Condição C2 (CPU Inteira):** Imposição da quota de 1,0 CPU, permitindo (teoricamente) a fluidez plena de uma thread solitária.

Em ambas as condições mantiveram-se restrições rigorosas padronizadas: teto de memória máxima de 256 MiB (com limite de swap idêntico a zero totalizando exatos 256 MiB de tolerância RAM+Swap), limite máximo de tarefas em 64 pids, ausência de conectividade ( `--network none` ), sistema de arquivos base somente leitura ( `--read-only` ), suspensão irrestrita de capacidades kernel ( `--cap-drop ALL` ), uso mandatário de utilizador sem privilégios (UID/GID 65534:65534) e ativação de `no-new-privileges` . Com o checksum gerado atrelado restritamente a N=20.000.000 e idêntico em todas as saídas verificadas (sha256 = `6140a0e6e3548422...` ), ficou cabalmente atestado que o processador realizou precisamente o mesmo esforço mecânico em todas as aferições independentemente das restrições transientes.

## 4.3. Consolidação de Resultados: Validação do Cgroup e Tempos de Resposta

É uma falácia presumir que comandos submetidos ao CLI do Docker são invariavelmente traduzidos para limites de hardware efetivos. Consequentemente, o limite real foi auditado diretamente nos arquivos de exposição do cgroup *dentro* da raiz do contêiner instanciado, descartando a presunção passiva dos flags passados ao daemon.

| Condição Testada | Leitura de `cpu.max` no cgroup | Leitura de `memory.max` e `swap` | Leitura de `pids.max` | Status dos Limites |
|---|---|---|---|---|
| **C1 (0,5 CPU Quota)** | `50000 100000` | Max: 268435456 (256 MiB). Swap: 0. | 64 | Confirmado nas 3 repetições |
| **C2 (1,0 CPU Quota)** | `100000 100000` | Max: 268435456 (256 MiB). Swap: 0. | 64 | Confirmado nas 3 repetições |

O limite de CPU, expresso estritamente no formato `50000 100000` , indica uma cota absoluta na qual os processos aninhados recebem um teto de 50.000 microssegundos de tempo computacional ativo a cada janela temporal contínua (período) de 100.000 microssegundos gerenciada pelo agendador CFS (Completely Fair Scheduler). Isso evidencia que o limite imposto ao contêiner em C1 atua por restrição de tempo agregado, e não obriga afinidade arquitetural, nem executa qualquer forma de alocação permanente ou reserva fixa de núcleo da CPU. As coletas resultaram em tempos de parede (*wall time*) brutos divergentes.

| Cond. | Wall Mediano (s) | CPU Processo (s) | Taxa (milhões it/s) | Eventos `nr_throttled` (Mediana) | Pico RAM no Grupo (MiB) | HWM do Processo (MiB) |
|---|---|---|---|---|---|---|
| **C1** | 5,201 | 2,600 | 3,845 | 52 acionamentos (de 52 a 53 períodos totais) | 12,3 a 12,5 | 21,2 a 21,3 |
| **C2** | 2,650 | 2,567 | 7,547 | 3 acionamentos marginais (em ~26 períodos) | 12,1 a 13,1 | 21,1 a 21,2 |

Nesta série analítica, verifica-se que a taxa de processamento sob 1,0 CPU (Condição C2) foi notadamente superior à da métrica sob restrição. Ao passo que a C1 lidou com ~3,85 milhões de iterações por segundo, a C2 galgou a taxa de 7,55 milhões it/s, materializando uma razão escalar de 1,96×, muito convergente à expectativa linear teórica da quota fracionada (esperava-se exatos 2,0). De suma importância é a coluna central referente a `nr_throttled`: na Condição 1, o cgroup reportou esgotamento orçamentário e contenção de estrangulamento da CPU em esmagadores 100% dos períodos observados (ex: 52 *throttles* sobrepostos a 52 janelas do agendador). Em contraste irrefutável, sob C2 (onde se dispunha teoricamente do núcleo inteiro de 1,0 CPU), a política registrou acionamentos erráticos flutuando entre 2 e 5 acionamentos residuais num oceano de cerca de 26 a 27 períodos completos, revelando que as breves contenções sofridas pela C2 advinham de ruídos corriqueiros da multiprogramação do hospedeiro, não de sanções da hierarquia cgroup do processo.

### 4.4. Menor Privilégio, Blindagem Lógica e Proteção de Mount Read-Only

Completando a orquestração do ambiente perimetral e testando os critérios de segurança de dados em trânsito e disco, configurou-se uma armadilha sintética no sistema de arquivos. Um diretório foi manufaturado na máquina host contendo o artefato de verificação `dados_demo/texto.txt` com conteúdo alfanumérico preestabelecido. Este caminho foi exportado à montagem do contêiner através do subcomando severo `--mount type=bind,source=SPWD/dados_demo,target=/dados,readonly`. Para assegurar que contornos de escalada via SUID/SGID falhariam por completo, manteve-se a restrição de UID 65534 e `--security-opt no-new-privileges`. Os logs capturados em tempo real expõem a execução mecânica das proteções do SO:

```
===== TESTE DE LEITURA =====
Documento sintetico da AV3
===== TESTE DE ESCRITA =====
sh: 6: cannot create /dados/tentativa.txt: Read-only file system
SUCESSO: escrita negada
```

A mensagem disparada pelo interpretador shell restrito foi precisamente categórica: `Read-only file system` (Falha sistêmica da operação por violação dos parâmetros do barramento de montagem lógicos do kernel), se diferenciando claramente da clássica repulsa superficial `Permission denied` de permissões POSIX. Isso indica inequivocamente que a camada primária de virtualização em nível de SO barrou de modo proativo a operação comissionada com base unicamente na arquitetura do namespace de montagem, neutralizando assim a possibilidade de o ataque sobrepujar, inclusive, políticas permissivas de arquivos do proprietário real que viessem a existir dentro da árvore de acesso.

## 5. Análise Detalhada (Respostas Analíticas do Experimento)

### 5.1. Onde está o kernel compartilhado e há interferência de Máquina Virtual?

O pilar estrutural que diferencia contêineres de hipervisores maduros reside no conceito de Kernel Compartilhado. Com base estrita na infraestrutura inventariada durante o experimento preparatório, o kernel subjacente utilizado não foi uma emulação gerada virtualmente. O sistema era o próprio hospedeiro físico: rodava ativamente o kernel Linux de variante principal 7.0.0-38-generic, gerenciado nativamente sobre as ferramentas do Ubuntu 24.04.2 em infraestrutura AMD. As medições físicas extraídas a partir do poderoso utilitário descritivo `lscpu` revelam que o sistema computacional opera no próprio metal e omite propositalmente quaisquer assinaturas sintéticas (como hipervisores KVM, Xen ou Hyper-V virtuais). Em que pese a identificação superficial da linha 'Virtualization: AMD-V', isto reflete exclusivamente uma habilidade adormecida baseada nos circuitos integrados da pastilha CPU física que autorizaria a criação futura de máquinas virtuais; contudo, nenhuma dessas VMs esteve ativamente posta em operação mediando este laboratório.

Isso responde taxativamente à dinâmica observada: os `sleeps` instanciados por intermédio da CLI docker não estão envelopados em uma bolha de sistema operacional enxuto; em vez disso, todos eles lançam chamadas de sistema (*syscalls*) e disputam preempção ombro-a-ombro perante as tabelas reais do mesmíssimo núcleo kernel Linux 7.0.0-38 que rege e despacha a máquina principal e sua interface gráfica integral. Não há um hipervisor e um sistema isolante convidado intercedendo e introduzindo lentidão; há apenas partições lógicas em tabelas nativas de memória e escalonamento.

### 5.2. A distinção dos handles do Namespace isolam plenamente um serviço?

A discrepância numérica obtida na coleta de inode PID via `/proc` mapeia com precisão empírica que os processos receberam ilusões e escopos independentes pelo kernel. Os handles apontam para instâncias exclusivas. Ao visualizar esse escopo estreito sob os olhos da primeira rotina iniciada dentro do contêiner, esta estrutura impõe que a rotina considere-se dona da raiz do PID 1, bem como que as tabelas adjacentes ( `mnt` e `net` ) pareçam intactas, exclusivas e puras, protegendo a aplicação de ser vista ou ver vizinhos operacionais de outros contêineres e aplicações corriqueiras.

No entanto, estas diferenças não demonstram garantias estritas ou mitigação global para recursos brutos físicos. A existência isolada de um PID diferente em nada constrange a quantidade infinita de ciclos do relógio da Unidade Lógica que podem ser demandados, ignorando severamente os picos anômalos de memória física (RAM). Um serviço encapsulado somente por namespaces sem a conjunção de Cgroups estaria habilitado para consumir predatoriamente a reserva integral da CPU física provocando gargalos hostis até causar travamento de infraestrutura vital (uma falha massiva de kernel panic por colapso das demais operações nativas do hospedeiro). Além disso, eles não avalizam um sanduíche de proteção de syscalls absolutas; uma técnica de bypass ou escape baseada na manipulação indevida do binário setcap conseguiria romper essas finas barreiras se o ID do root verdadeiro ainda estiver retido sem o remapeamento adequado via namespace *user*.

### 5.3. Qual é a correlação exata entre o estrangulamento (Throttling) e as flutuações da Mediana de Trabalho?

Ao se esmiuçar a matemática estatística gerada pela matriz de processamento contínuo PBKDF2, a mediana representativa correspondente à C1 e C2 evidenciou a razão final calculada da experimentação empírica (1,96), uma divergência pequena em relação à modelagem estatística exata de cgroups puramente lineares (esperava-se exatos 2,0).

É crucial notar a discrepância de amplitudes em relação às variâncias documentadas. Paradoxalmente, C2 (1,0 CPU irrestrita) exibiu uma oscilação na amplitude (a variação máxima frente ao desempenho mínimo mediano) significativamente maior que C1, marcando 6,4% de amplitude temporal contra meros 2,8% computados em C1. Esse fato, aliado à premissa de que a terceira repetição consecutiva em ambas condições decaiu acentuadamente a sua agilidade processual (+2,8% de latência excedente em C1, e uma degradação de +3,4% em C2 relativamente à tentativa pioneira correspondente), denuncia que o problema transcende a capacidade gerencial direta do grupo de limitação. A variação intrínseca na resposta dos contêineres e sua discrepância no ganho linear é amplamente atribuível à **principal ameaça à validade técnica deste experimento**.

O fator que contamina as medições decorre primordialmente da seleção do hospedeiro utilizado. Trata-se de uma máquina estilo *desktop multimídia* nativa em uso contínuo severo, com expressivos 5,1 GiB de sua memória ocupados de forma concorrente em repouso por processos adjacentes não controlados e sob a regência severa de um governador de políticas de eficiência energética (escalonamento inteligente e agressivo de *Boost* das frequências centrais entre 421 MHz ocioso até explosivos e térmicos 4668 MHz). Sem estampar a frequência momentânea cravada durante as medições para equalizar as métricas puristas ou controlar variáveis térmicas no núcleo centralizado durante cada pico, e usando cargas singelas focadas numa base de solitária amarrada sem o controle de alinhamento com a escalada geral do micro-código em segundo plano, os ruídos interagem diretamente e afetam severamente as janelas puras propostas para a aferição técnica das restrições de escalonamento implementadas no Linux por meio dos cgroups. Não constituem erros analíticos do subsistema de contêineres, mas sim limites observacionais severos oriundos da contaminação fisiológica originada da infraestrutura de hospedagem da qual a amostragem se embasou.

*thread*

## 6. Implicações Estratégicas em Ambientes Complexos: Segurança na Implantação de LLMs (RAG)

A arquitetura e os achados fundamentais demonstrados de forma sintética neste ensaio ganham escopos imensos de proteção crítica, mitigação e blindagem no momento em que transicionamos esse laboratório para um servidor de produção contendo motores pesados de inferência artificial. Quando o provedor aloca processos sofisticados para servir modelos generativos massivos e rotinas baseadas na estruturação profunda RAG (Retrieval-Augmented Generation), o uso e as imposições demonstradas não são apenas desejáveis, mas compulsoriamente vitais para a infraestrutura moderna do centro de processamento.

Um contêiner destrancado gerando *tokens* em altíssima velocidade demanda fluxos intensos de dados da arquitetura, e na ausência compulsória de uma trava `memory.max`, pequenas falhas estruturais, chamadas de vetores circulares falhas em manipulação tensorial pesada ou até picos imprevisíveis na quantidade infame de requisições paralelas fariam um único contêiner esgotar, letalmente, toda a memória volátil física vital pertencente ao nó hospedeiro. Isso levaria fatalmente o subsistema nativo do host Linux a invocar arbitrariamente o algoritmo cirúrgico indiscriminado de assassinato de contingência denominado agressivamente de OOM Killer (*Out-of-Memory Killer*), encerrando não apenas as solicitações excedentes do serviço abusivo culpado, como correndo risco catastrófico de aniquilar processos periféricos e matar brutalmente serviços justos paralelos vitais para a saúde da orquestração global no *cluster* unificado, ferindo princípios base de convivência de multilocatários. Além disso, a aplicação combinatória de `pids.max` paralisa a execução indiscriminada de ataques massivos de *fork bomb* orquestrados para minar a numeração da tabela, isolando comportamentos criminosos intencionais ou acidentais provenientes das injeções não controladas das filas sistêmicas e dos agendamentos pesados do hardware.

O laboratório documenta que o encapsulamento físico de um diretório ou repositório montado restritamente usando *mounts read-only* garante imunidade sistêmica irrefutável (proteção comprovada que negou escritas independente de identidades e privilégios intrínsecos de PID na sessão 4.4). Em uma arquitetura central RAG para modelos generativos maduros que carrega a base primária de respostas fundamentadas via conhecimento documental, montar estes *corpora* (documentos estruturados primários, guias organizacionais confidenciais, definições técnicas irrestritas) de modo absolutamente restrito protege e estanca vulnerabilidades assustadoras relacionadas a envenenamento e manipulação lógica arbitrária de pacotes (*model poisoning* e inserções hostis nas premissas). Caso ocorresse uma evasão brutal ou exploração da injeção arbitrária no interpretador backend (como uma falha fatal numa injeção avançada RCE executada em Python que controla o fluxo generativo do host contêiner), a estrutura subjacente *read-only* atuaria instantaneamente paralisando a mutação das instruções confidenciais da organização, salvaguardando a neutralidade estatística gerencial subjacente.

Embora tais mecanismos abafem incidentes cataclísmicos orçamentários, **riscos profundos estruturais e críticos perduram silenciosamente**: A estrutura inteira foi evidenciada ancorada sob um núcleo aglutinado (kernel 7.0.0 nativo compartilhado). Se uma brecha fatal crítica (CVE dia zero) vier a ser localizada expondo a interface exposta interna ou permitindo transposição nas camadas das próprias chamadas originais e nativas operadas pelo Cgroup subjacente ou manipulação impura dos barramentos sensíveis abertos remanescentes e subpastas de leitura do diretório sistêmico `/proc` ou o acoplamento virtual em diretórios de `/sys`, o agressor poderá escalar suas permissões lateralmente ao status onipotente do próprio proprietário principal da máquina. Da mesma maneira profunda, Cgroups de controle padrão de Linux expurgam falhas para ciclos nativos da memória de acesso (RAM livre do sistema local), todavia não regem ou dominam, de maneira padronizada e nativa isolada, picos ou quotas limitantes para acesso desenfreado das placas massivas compartilhadas dedicadas em cálculos numéricos paralelos de vetores avançados (placas massivas VRAM nativas operando paralelamente via passagens complexas da arquitetura *PCI-express*, usualmente orquestradas exclusivamente pelas ferramentas e instâncias de terceiros acopladas diretamente como o complexo NVIDIA Container Toolkit, as quais gerenciam esses dispositivos discretos contornando Cgroups locais básicos das bibliotecas clássicas de limitação de memória Linux).

## 7. Limitações Críticas, Procedimentos de Contingência e Protocolos de Segurança

Para atuar preservando rigorosamente e irrefutavelmente a disponibilidade contínua dos provedores sistêmicos e prevenir as falhas de bloqueios perigosos da infraestrutura no decorrer de experimentações intensas atreladas à produção do relatório matriz base, adotaram-se as diretrizes de segurança extremas: Jamais injetaram-se quaisquer rotinas intencionais voltadas ao bloqueio operacional (nunca incitaram-se picos de OOM acionáveis e nenhuma emulação criminosa baseada nas exaustões típicas dos ataques estruturais do modelo de expansão exponencial algorítmica conhecidos amplamente como *Fork Bombs* em scripts destrutivos paralelizáveis). Paralelamente, foram abandonados comandos manipulatórios baseados na alteração manual e na interferência arriscada ou sobreposições imperativas perigosas e diretas ao algoritmo do *scheduler* nativo ou inserções grosseiras diretas de tempo real impostos na pasta protegida no escopo sistêmico do diretório crítico em `/sys`.

Controles preventivos robustos foram acoplados aos scripts de verificação operacionais: além das diretrizes internas estabelecidas do programa base limitando ativamente a continuidade da carga contida nativa ao redor do timer máximo impositivo acoplado após cerca de 60 segundos por limite algorítmico, o *wrapper* subjacente atrelado aciona os mecanismos severos interligados através de alarmes UNIX globais externos que matam as operações bloqueantes do contêiner via *GNU Timeout* em 90 segundos fixos, certificando e salvando indubitavelmente a sanidade irrestrita das preempções gerais no núcleo da placa computacional principal perante condições acidentais adversas em *deadlocks* catastróficos eventuais.

Todos os limites operacionais estritamente confirmados (tais como limites em ramificações puras nas marcações das predefinições `memory.peak` gravitando as flutuações minúsculas constantes marcando entre cerca de 12,1 a estritos 13,1 MiB de processamento ativo para um teto colossal imensamente superestimado estático que orbitava restrições limites de 256 MiB máximos preestabelecidos; ou mesmo a predefinição fixada para limite irrestrito do uso das tarefas conjuntas do processo) nunca foram exercitados à máxima pressão letal nas exaustões extremas contidas pela documentação, em reverência e atenção rígidas aos avisos diretos oriundos expressamente expostos no enunciado mestre matriz proibitivo do uso de OOM deliberado em sistemas unificados sem autorização institucional ampla, o que converte os argumentos fundamentais associados de proteção das defesas em presunções extraídas essencialmente na pura observação das documentações originais oficiais dos criadores do Docker engine global integrado, sem as medições empíricas extremas destrutivas baseadas nestas amarras protetivas exclusivas.

## 8. Aplicação de IA Generativa na Elaboração dos Dados

A arquitetura e os modelos adaptados na metodologia do laboratório de orquestração de scripts complexos atrelados originados nos conceitos das tarefas pregressas das Atividades base relativas às inferências da disciplina anterior envolveram intervenção estruturada de engenharia de software automatizada auxiliada. A ferramenta base e o mecanismo motor de linguagem abrangente selecionada operou usando especificamente as infraestruturas lógicas acopladas ao modelo *Claude Sonnet versão 5.0*.

O direcionamento dos fluxos e suas atuações envolveram primariamente o mecanismo de reescrita padronizada e limpeza sintática de refatoramento no código fonte nativo do arquivo em Python original de orquestração matemática de criptografia contínua iterativa acoplada no núcleo `PBKDF2` de cargas passadas base das execuções em medições brutas cronológicas em lote, convertendo as saídas confusas da interface terminal e estruturando os armazenamentos e despejos tabulares persistidos no padrão universal da infraestrutura descritiva unificada orientada de dados em arquivos analíticos (formatações em saídas JSON consistentes, limpas, auditáveis padronizadas para análise cruzada posterior base analítica automatizadora consolidada de leitura no *resumo.md* final). Todos os retornos foram devidamente escrutinados rigorosamente no código matriz antes do comprometimento com o experimento em produção base final (hash oficial manifestado do arquivo validador do pacote em `b1165487d16ee...`) impedindo injeções errôneas analíticas nos dados das coletas reais brutas oriundas da atividade contínua final.

## 9. Referências Técnicas


- Linux man-pages. **namespaces(7)**. Disponível em: https://man7.org/linux/man-pages/man7/namespaces.7.html
- Linux Kernel Documentation. **Control Group v2**. Disponível em: https://docs.kernel.org/admin-guide/cgroup-v2.html
- Docker. **Resource constraints**. Disponível em: https://docs.docker.com/engine/containers/resource_constraints/